Unipampa - Bagé

# TP1
# ALGORITMOS DE CLASSIFICAÇÃO/ ORDENAÇÃO


Apresentado por:

THALES KUN

BRUNO VIEIRA

# Sumário

01. Problema e objetivos

02. Algoritmos implementados

03. Metodologia

04. Resultados

05. Análise

06. Conclusões

# Problema

Algoritmos de ordenação são fundamentais em computação, mas sua eficiência varia enormemente conforme o tamanho e a ordem dos dados.

# Objetivo

Implementar 10 algoritmos de ordenação em Java

Verificar corretude em 87 cenários distintos

Medir desempenho em 4 tamanhos × 3 ordenações de entrada

Comparar tempos reais com complexidades teóricas

Identificar quando cada algoritmo é mais adequado

# Configuração das máquinas

Os tempos apresentados nos próximos slides serão comparados entre estas duas configurações.

| Especificação | Computador 1 | Computador 2 |
|---|---|---|
| **Processador** | Intel Core i5-1135G7 | Ryzen 7 5700G |
| **Memória RAM** | 8 GB | 16 GB |
| **Sistema operacional** | Zorin OS 18.1 | Ubuntu via WSL |
| **Versão do Java** | Java 25.0.2 | Java 25.0.2 |

**A análise dos resultados considera o desempenho relativo entre as duas máquinas.**

Unipampa - Bagé

# 10 algoritmos de ordenação implementados

Comparação por complexidade assintótica

**7 por comparação**

| Algoritmo | Complexidade |
|---|---|
| Bubble | $O(n^2)$ |
| Insertion | $O(n^2)$ |
| Selection | $O(n^2)$ |
| Shell | Depende dos gaps |
| Heap | O(n log n) |
| Merge | O(n log n) |
| Quick | O(n log n)* |

**3 não comparativos**

| Algoritmo | Complexidade |
|---|---|
| Counting | O(n + k) |
| Radix | O(d(n + b)) |
| Bucket | O(n + k)* |

*Quick: caso médio; pior caso $O(n^2)$.

*Bucket: custo esperado sob distribuição adequada dos dados.

**Objetivo: comparar o crescimento teórico com os tempos medidos experimentalmente.**

# Metodologia

**Tamanho dos vetores**
1000 · 10000 · 100000 · 1000000

**Ordenações de entrada**
Crescente · Decrescente · Aleatório

**Repetições por configuração**
30 Repetições (n ⩽ 100k) · 10 Repetições (n = 1mi)

OBS: No computador 1, foram medidos 10 repetições com 100 mil elementos

# Estrutura dos algoritmos

## Algoritmos por comparação (7)

- Vetores Integer[]
- Métodos genéricos com Comparable<T>
- Comparações realizadas com compareTo()

## Algoritmos não comparativos (3)

- Vetores int{}, com valores não negativos
- Counting: utiliza contagens
- Radix: trabalha com dígitos
- Bucket: distribui elementos em baldes

# Como os tempos foram comparados?

Mesmo procedimento experimental

Mesmos valores, tamanhos e organizações de entrada

Cópias independentes para cada execução

Medição apenas do método de ordenação

CUSTOS INCLUÍDOS NA MEDIÇÃO

**Counting: criação e processamento do vetor de contagem**

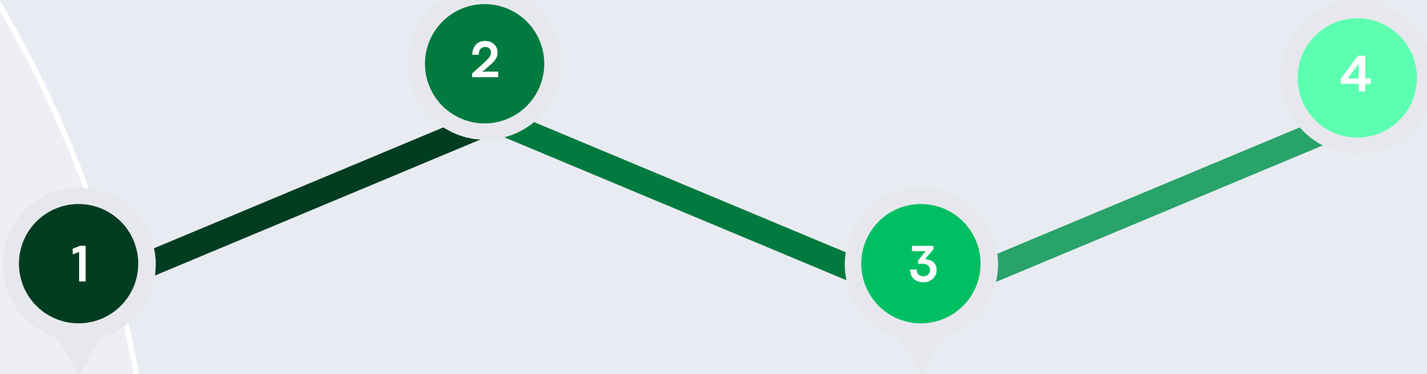
**Radix: criação de estruturas auxiliares e passadas pelos dígitos**

**Bucket: criação, distribuição e ordenação dos baldes**

**Limitação: as diferenças entre Integer[] e int{}, além das características dos algoritmos, impedem considerar os tempos entre os dois grupos como perfeitamente equivalentes.**

# Medição de desempenho

Pipeline por execução




**Warm-up da JVM**

3 execuções preliminares descartadas para reduzir efeitos da inicialização e otimização.

**Cópia do vetor original**

Cada execução recebe os mesmos valores, sem incluir o clone no tempo medido.

**Medição com nanoTime()**

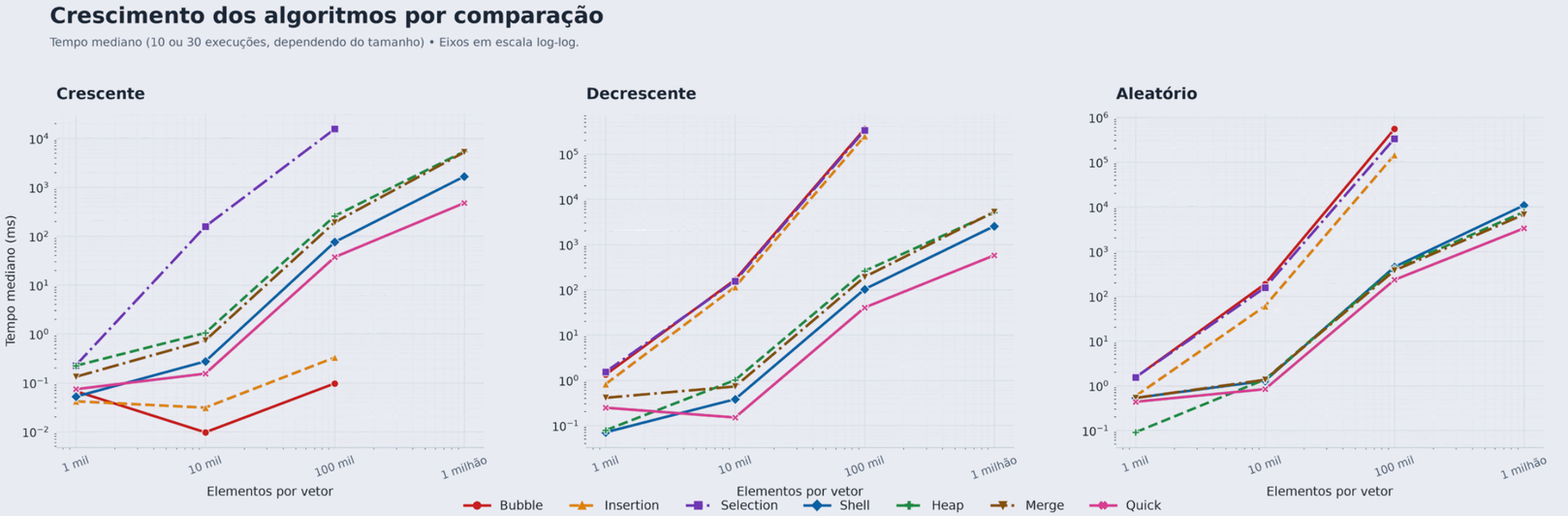
Tempo = fim − início. Apenas a execução do algoritmo é cronometrada.

**Registro no arquivo CSV**

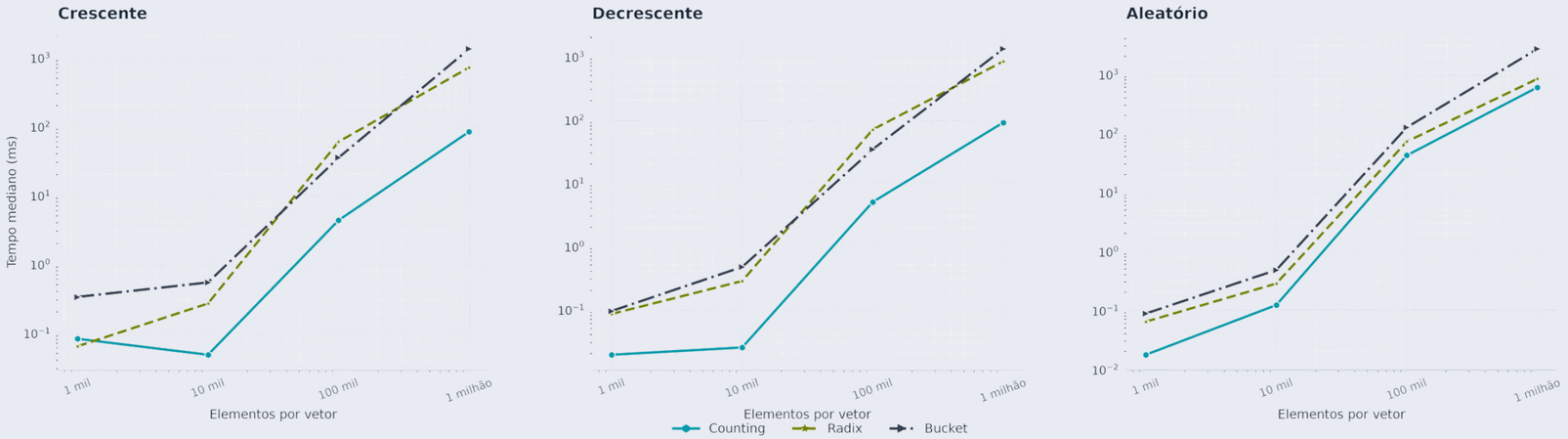
Cada tempo é salvo em nanossegundos, com flush do buffer após o registro.

Unipampa - Bagé

# Gráfico - C1

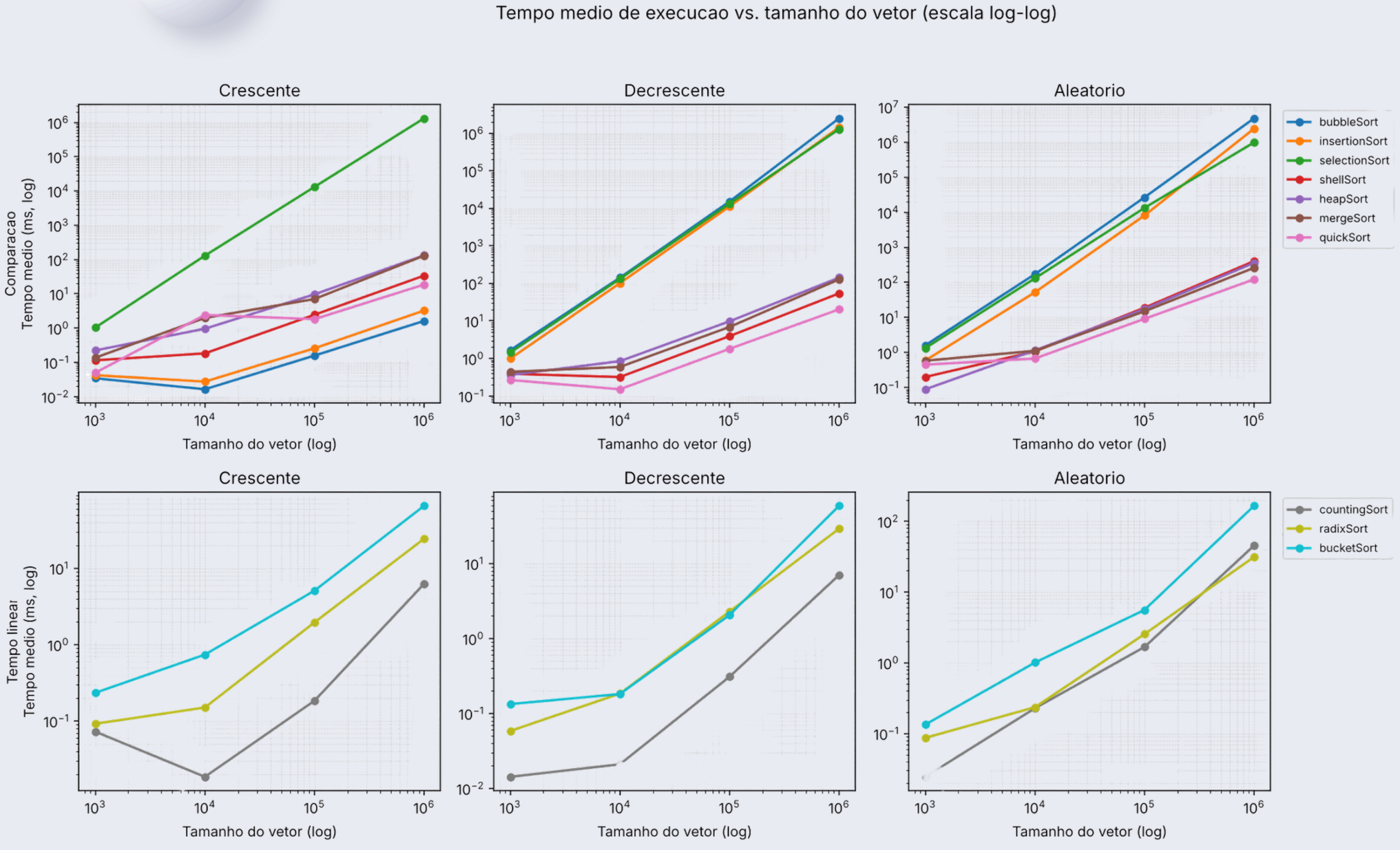



Curvas terminam no último tamanho realmente medido. Sem ponto significa teste não realizado.




Curvas terminam no último tamanho realmente medido. Sem ponto significa teste não realizado.

# Gráfico - C2


Tempo medio de execucao vs. tamanho do vetor (escala log-log)
Crescente
Decrescente
Aleatorio
Comparacao
Tempo medio (ms, log)
Tamanho do vetor (log)
bubbleSort
insertionSort
selectionSort
shellSort
heapSort
mergeSort
quickSort
Crescente
Decrescente
Aleatorio
Tempo linear
Tempo medio (ms, log)
Tamanho do vetor (log)
countingSort
radixSort
bucketSort

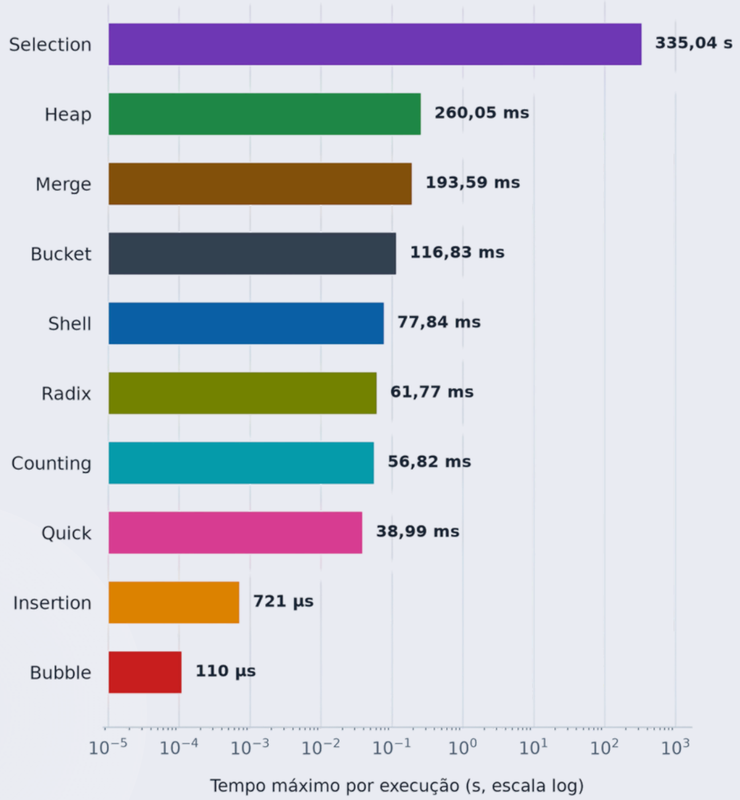
Unipampa - Bagé

# Gráfico - C1

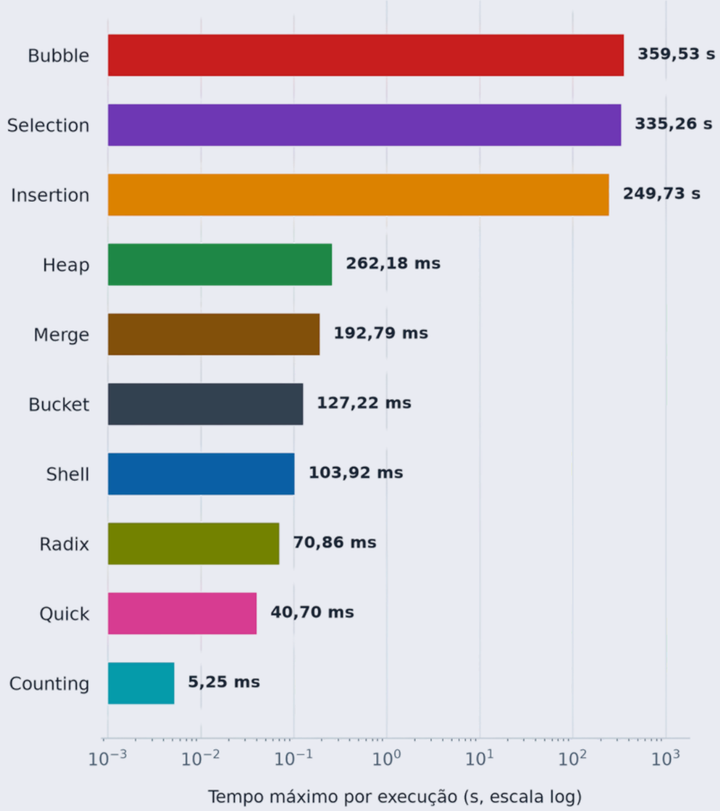
## 100 mil elementos | Maior tempo registrado por algoritmo

Uma barra por algoritmo, mostrando o MAIOR tempo entre as 10 execuções de cada entrada.

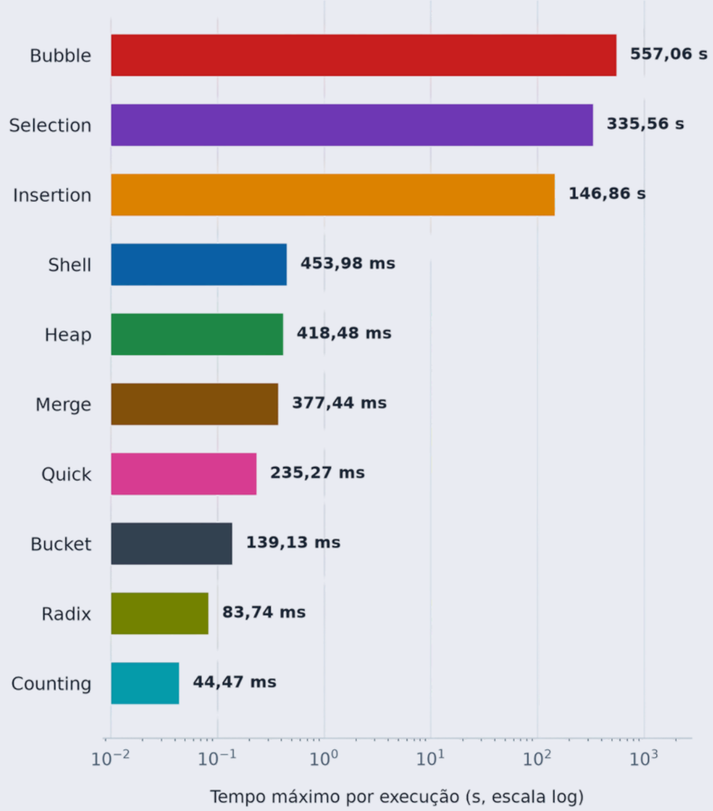
### Crescente




### Decrescente




### Aleatório




Escala logarítmica em segundos: leia os valores nas barras; o comprimento NÃO é proporcional ao tempo.

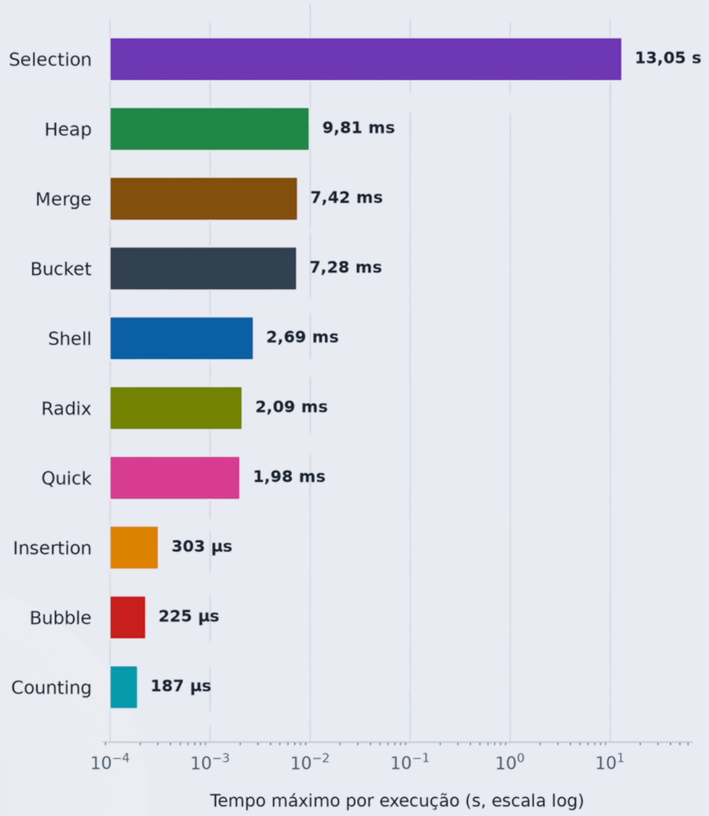
Unipampa - Bagé

# Gráfico - C2

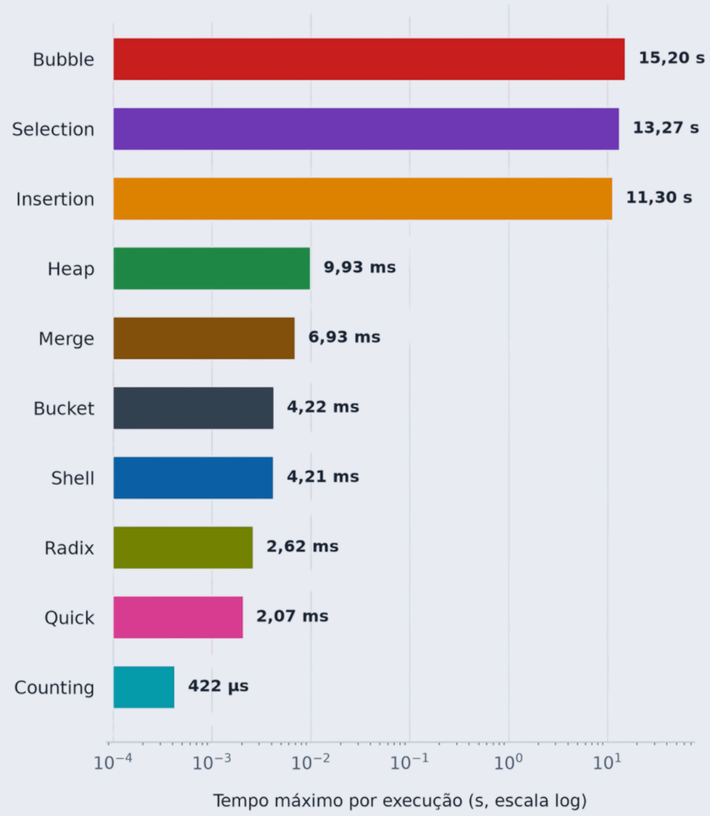
## 100 mil elementos | Maior tempo registrado por algoritmo

Uma barra por algoritmo, mostrando o MAIOR tempo entre até 10 execuções de cada entrada.

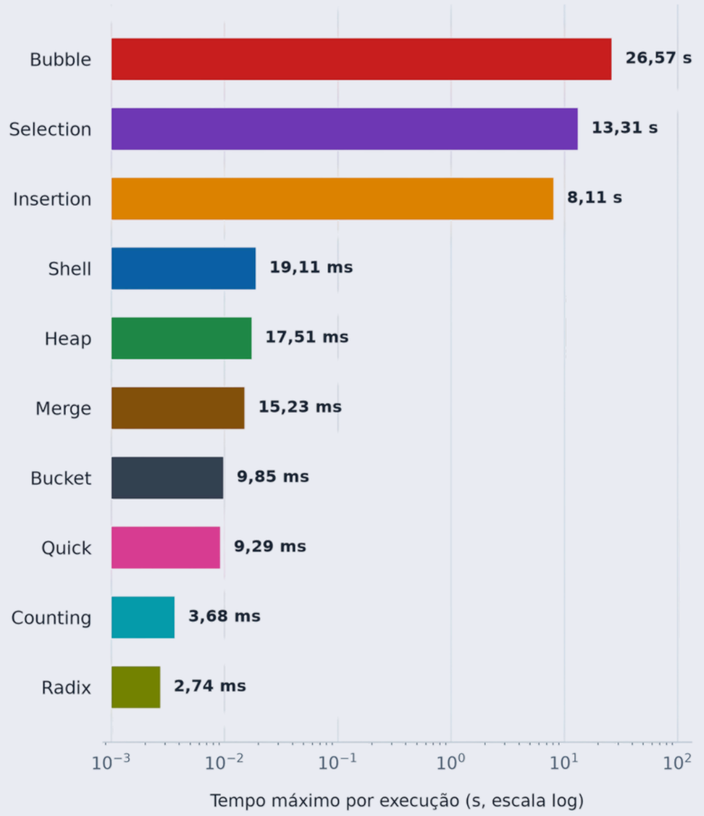
### Crescente




### Decrescente




### Aleatório



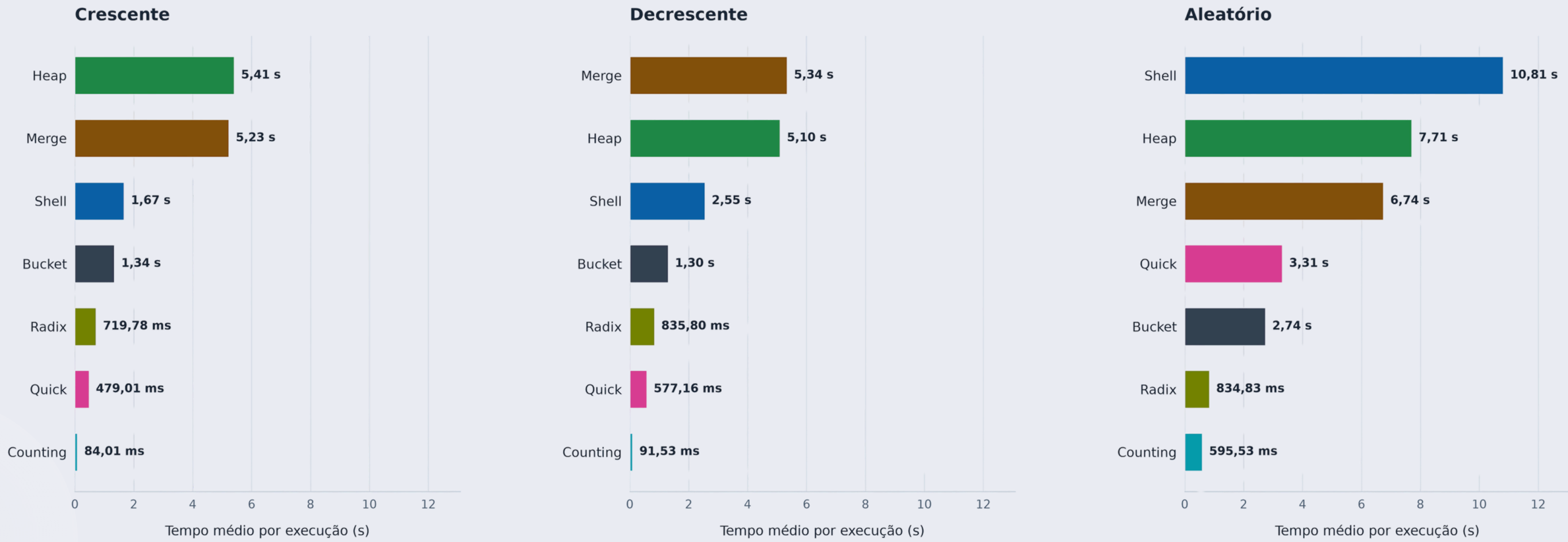

Escala logarítmica em segundos: leia os valores nas barras; o comprimento NÃO é proporcional ao tempo.

Unipampa - Bagé

# Gráfico - C1

## 1 milhão de elementos | Tempo médio dos algoritmos executados

Média das 10 execuções por configuração • Barras horizontais em escala LINEAR comum.



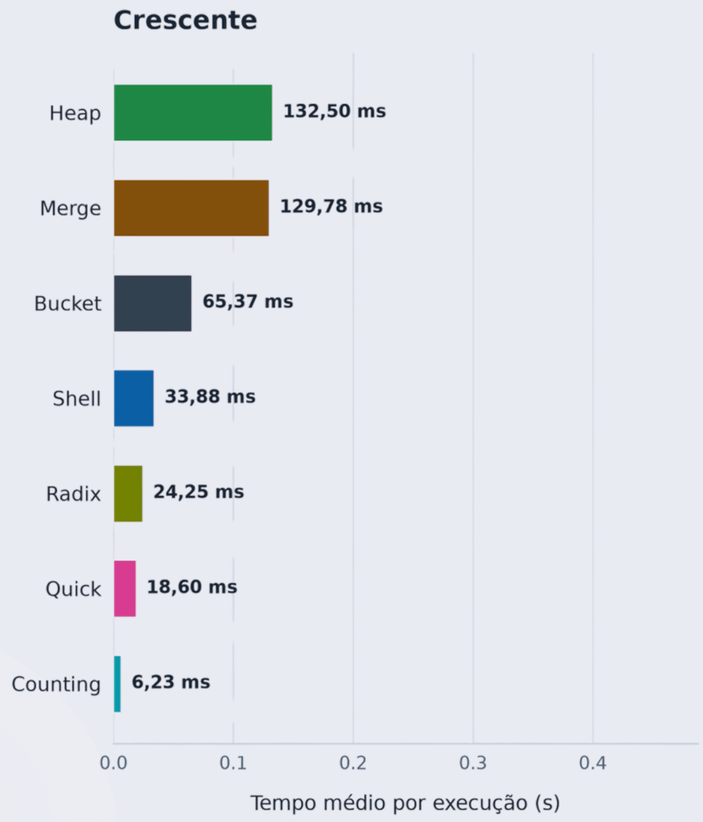

Bubble, Insertion e Selection não foram executados em 1 milhão de elementos.

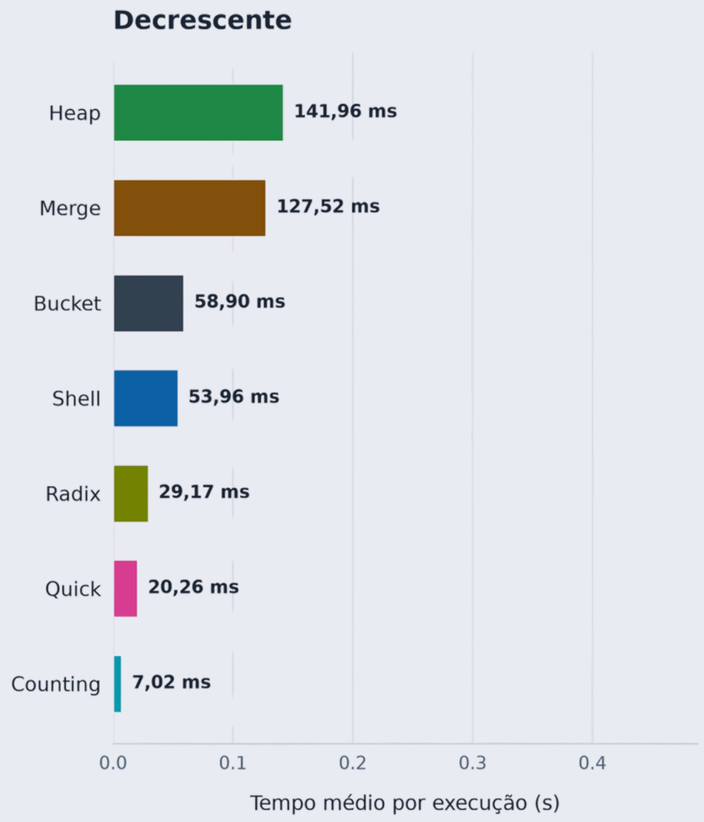
Unipampa - Bagé

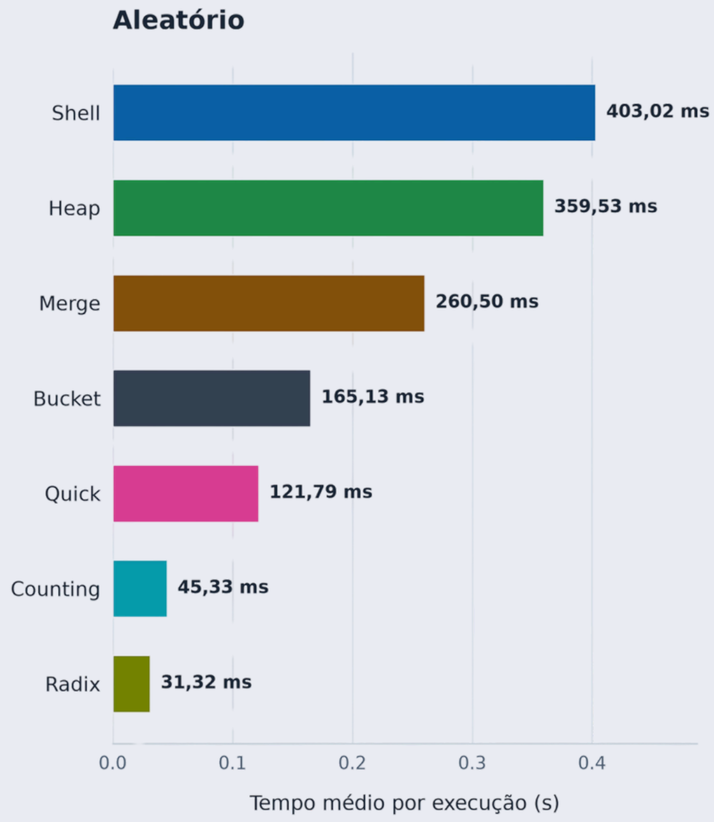
# Gráfico - C2

## 1 milhão de elementos | Tempo médio dos algoritmos rápidos

Média de até 10 execuções por configuração • Barras horizontais em escala LINEAR comum.



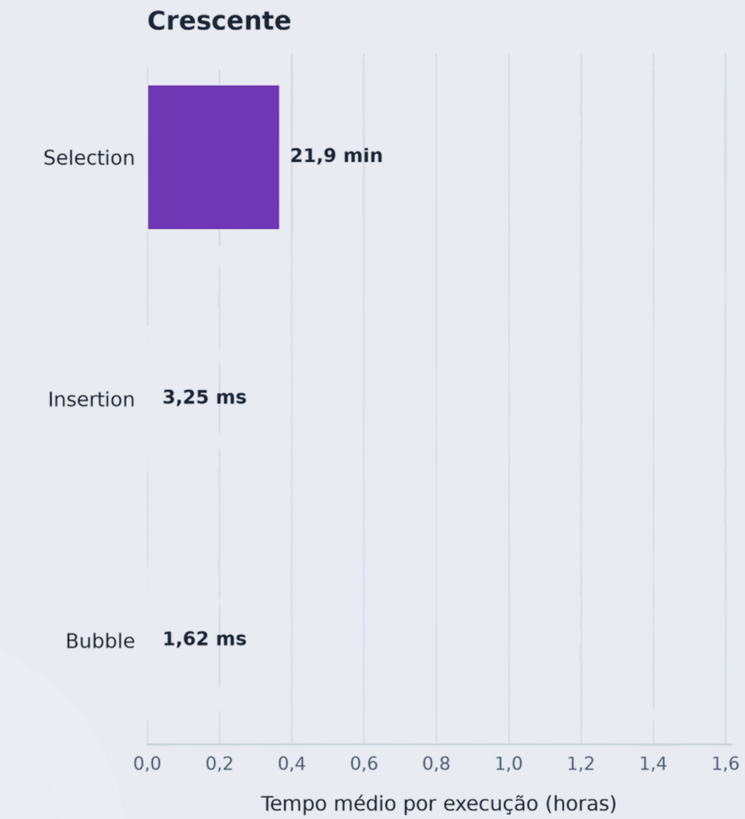

Bubble, Insertion e Selection (quando medidos) aparecem separadamente no gráfico 07.

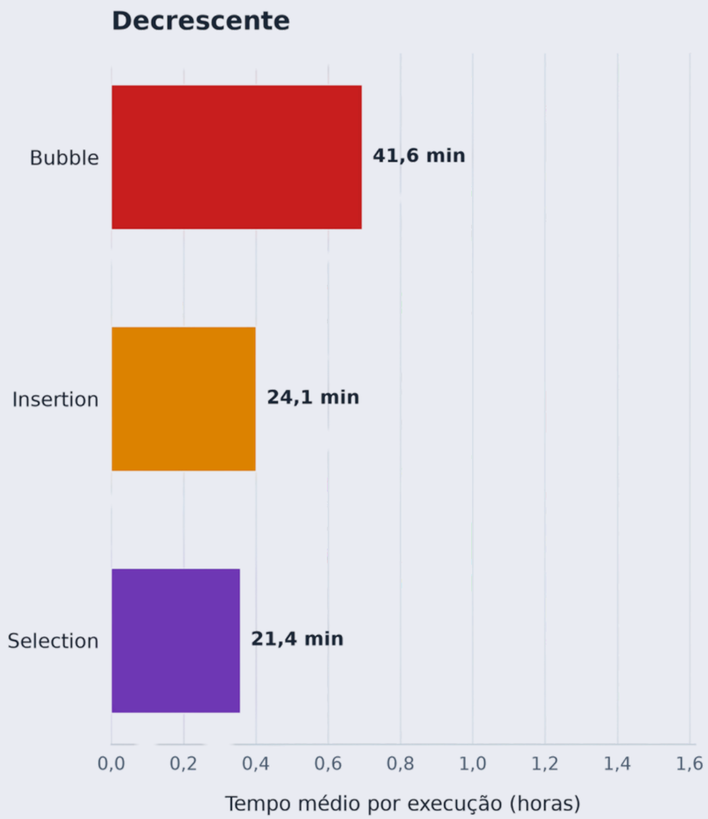
Unipampa - Bagé

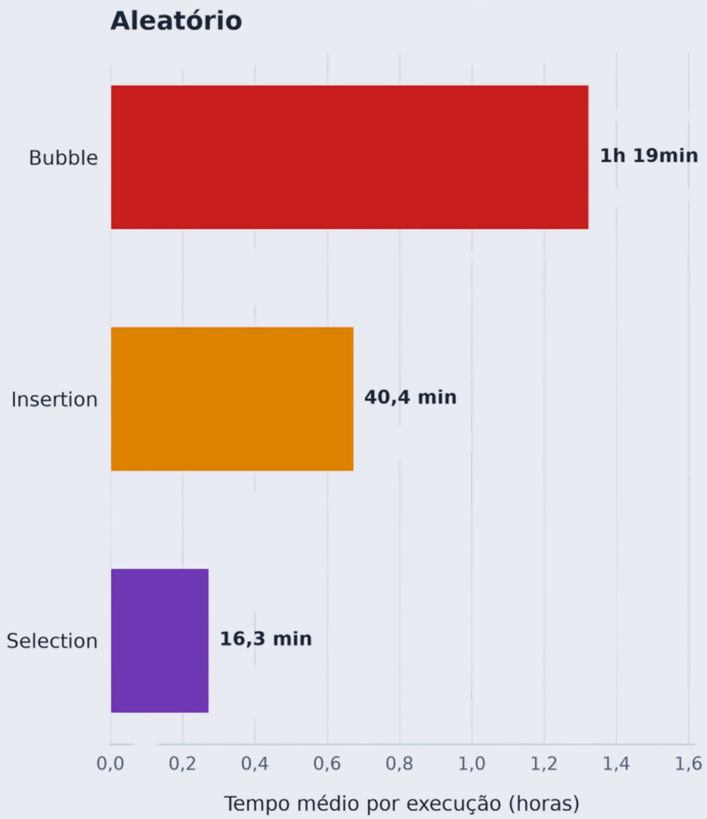
# Gráfico - C2

## 1 milhão de elementos | Bubble, Insertion e Selection

Tempo médio de até 10 execuções por entrada • Barras lineares, eixo em HORAS.

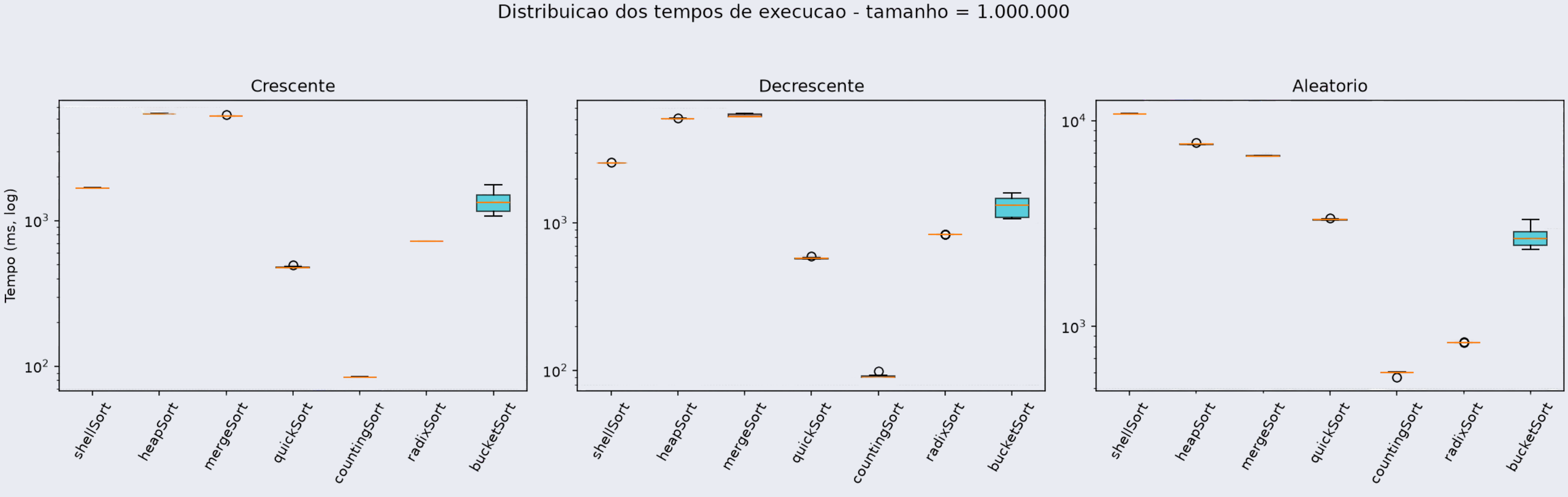


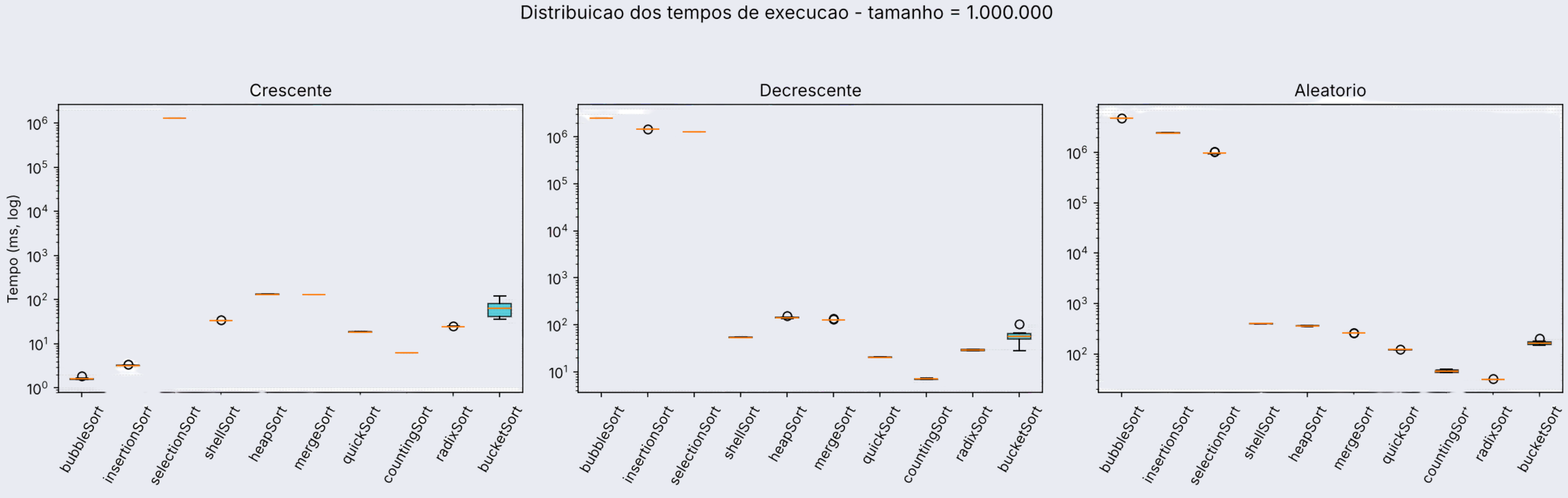




Tempos muito pequenos ficam quase invisíveis nessa escala; leia também os rótulos (h/min/ms).

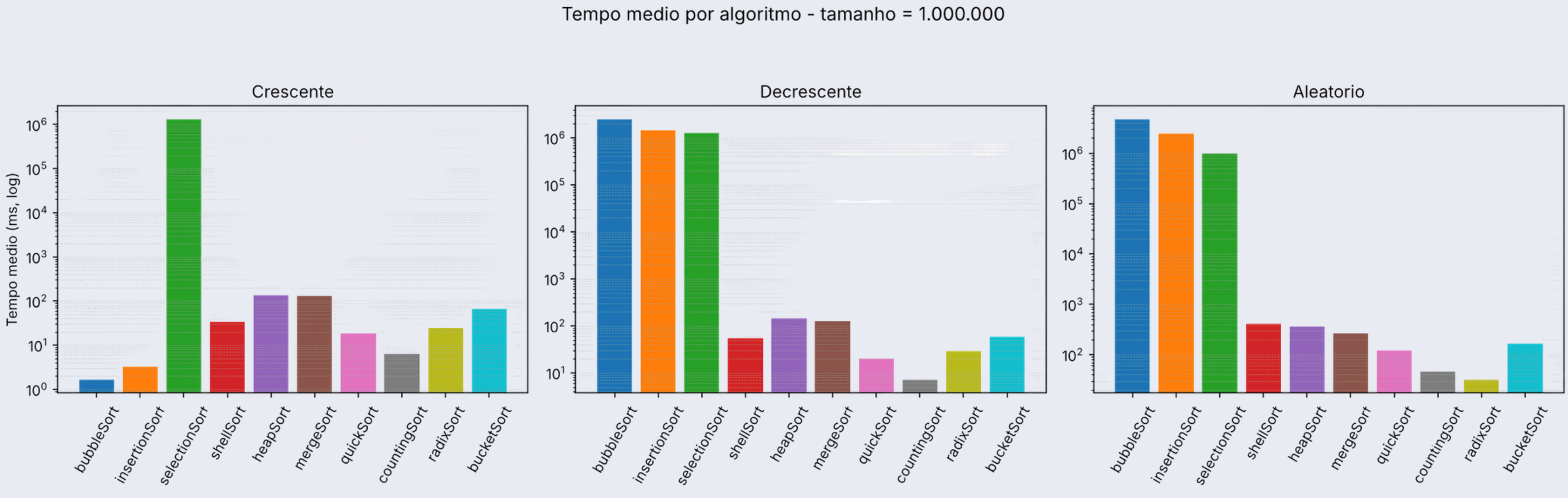
# Boxplot - C1


Distribuicao dos tempos de execucao - tamanho = 1.000.000
Crescente
Decrescente
Aleatorio
Tempo (ms, log)
$10^2$
$10^3$
$10^4$
shellSort
heapSort
mergeSort
quickSort
countingSort
radixSort
bucketSort

# Boxplot-C2


Distribuicao dos tempos de execucao - tamanho = 1.000.000
Crescente
Decrescente
Aleatorio
Tempo (ms, log)
bubbleSort
insertionSort
selectionSort
shellSort
heapSort
mergeSort
quickSort
countingSort
radixSort
bucketSort

# Gráfico  Completo - C2



# Impacto da ordenação de entrada

**Insertion Sort — sensível à ordenação**

O(n) — vetor já ordenado, zero trocas

O($n^2$) — inversão total

**Selection Sort — imune à ordenação**

Sempre percorre todo o vetor para achar o mínimo — O($n^2$) em todos os casos. Confirmado nos dados: tempos quase idênticos para os 3 tipos de entrada.

**Quick Sort — pivô central evita o pior caso**

O pivô central é a mediana em vetores ordenados, então as partições ficam balanceadas: ~20 ms com n = 1M.

# Impacto da ordenação de entrada

### Merge Sort e Heap Sort — consistentes

Garantia O(n log n) em todos os casos — confirmado experimentalmente: variação <10% entre crescente, decrescente e aleatório.

### Shell Sort — surpreendentemente bom

Fator de crescimento medido (~78×) ficou abaixo do esperado para O(n log² n). Beneficia-se de entradas parcialmente ordenadas.

### Algoritmos lineares — dominados por k

Counting Sort: range = 10 × n → k = 10.000.000 para n = 1M. Radix Sort mitiga isso com múltiplas passadas, ficando mais próximo do teórico.

# Limitações

Acabamos tendo uma limitação no computador 1
por conta de demora para executar com 100 mil elementos, foram retirados da execução do 1 milhão:

- Bubble Sort
- Insertion Sort
- Selection Sort

No computador 2 foram mantidas as execuções para termos uma visão ainda melhor de como se portaram os algoritmos mais demorados.

# Conclusões

### A teoria prevê as tendências, mas não os valores absolutos

Fatores de cache, cópia de objetos boxed (Integer[]) e overhead da JVM afastam os tempos reais das curvas ideais.

### Bubble Sort é 2,8× mais lento que o esperado em dados aleatórios

O acesso desordenado à memória destrói a localidade de cache, amplificando o custo quadrático.

### Quick Sort vence na prática

Melhor uso de cache que o Merge, fator ~170× próximo do teórico. O pivô central evita degenerar em entrada ordenada.

### Shell Sort superou as expectativas teóricas

Fator de crescimento medido ~78× (esperado ~150×). Boa escolha quando Merge/Quick não podem ser usados.

### Radix Sort é o linear mais consistente

Fator ~129× vs. Counting Sort ~248×. Múltiplas passadas evitam o problema do range k grande.

### Recomendação prática

Use Quick Sort para uso geral · Merge Sort quando estabilidade importa · Radix Sort para inteiros em larga escala · evite $O(n^2)$ acima de 100.000 elementos.

Unipampa - Bagé

# OBRIGADO!

Github do projeto:
https://github.com/ThalesKun/classificacao-tp1